Lab06 - ADC

1. Cos'è un ADC?

L'**ADC (Analog-to-Digital Converter)** è il componente che permette al microcontrollore di leggere una **tensione analogica** e trasformarla in un **numero digitale** che il programma può elaborare.

Per esempio, supponiamo di avere un sensore che produce Vin=1.65V. Il microcontrollore non può lavorare direttamente con "1.65 V" come valore numerico. L'ADC converte questa tensione in un numero, ad esempio ADC=2048, se stiamo usando un ADC a 12 bit (4096 livelli) con riferimento di 3.3 V.
Il percorso è quindi:

Grandezza fisica → Sensore → Tensione analogica → ADC → Numero digitale → Programma

2. Risoluzione dell'ADC

Una delle caratteristiche più importanti è la **risoluzione**, espressa in bit. Un ADC a **N bit** può rappresentare 2^N livelli diversi. Nel caso tipico di un ADC STM32 a **12 bit**, il risultato va da $0 \rightarrow 4095$, perché ci sono $2^{12} = 4096$ livelli.

Quindi:

- 0 V → circa **0**
- 3.3 V → circa **4095**

NB: anche se l'ADC è a 12 bit, comunque i valori vengono riconvertiti in 16bit perché i nostri registri sono da 16 bit.

3. Come si passa dalla tensione al numero?

La relazione ideale è:

$$ADC_{Value} = \frac{V_{In}}{V_{Ref}} \cdot (2^N - 1)$$

dove:

- V_in = tensione applicata all'ingresso ADC
- V_Ref = tensione di riferimento
- N = numero di bit
- ADC_Value = valore digitale prodotto dall'ADC

4. E viceversa: dal numero alla tensione

Molto spesso nel programma abbiamo il valore ADC e vogliamo conoscere la tensione. Invertendo la formula:

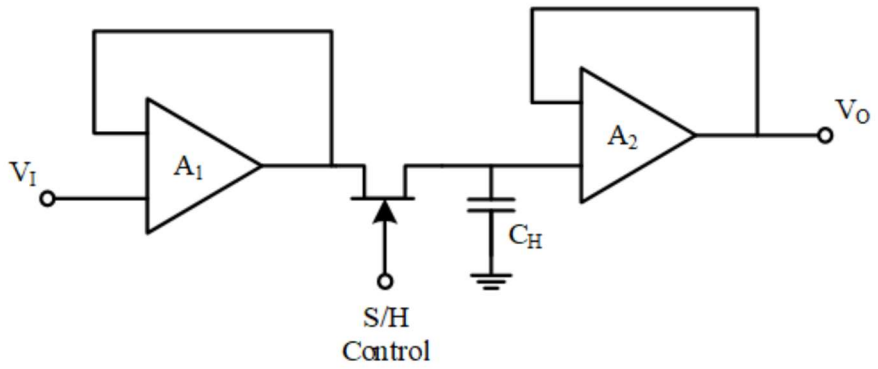
$$V_{In} = \frac{ADC_{Value} \cdot V_{Ref}}{(2^N - 1)}$$

5. Cosa succede fisicamente nell'ADC?

L'ADC fa essenzialmente tre operazioni:

1. Sampling

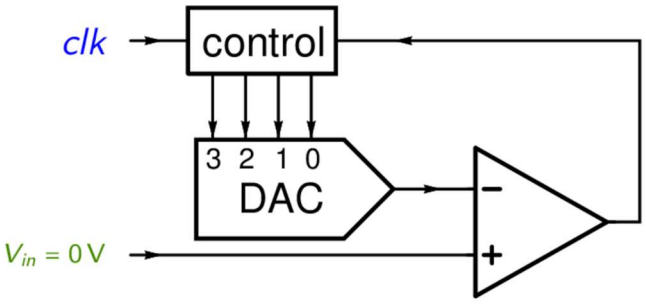
L'ADC **campiona** la tensione analogica. Significa che prende il valore della tensione in un determinato istante.



- **Fase di Sample**: leggiamo il segnale
  - Lo switch è chiuso
  - Il segnale carica C
  - NB: la velocità con cui V_c segue V_in dipende dalla tau del circuito RC (ricorda il Teorema di Shannon).
- **Fase di Hold**:
  - Lo switch si apre
  - Leggiamo il valore raggiunto da V_c

2. Conversione

Il valore analogico viene confrontato con i livelli disponibili dell'ADC. Un ADC a 12 bit ha 4096 livelli possibili.
In un SAR, il valore viene confrontato con i diversi livelli fino a trovare quello di cui fa parte. Partiamo dal livello MSB, se V_c sta sopra scriviamo 1 al MSB, altrimenti 0, e così via, fino a scrivere tutti i bit della codifica, dal MSB al LSB.



NB: questo processo prende tempo, perché dobbiamo fare 12 comparazioni (12 colpi di clock) prima di avere il valore digitale definitivo.

3. Quantizzazione

La tensione viene associata al livello digitale più vicino. Quindi, invece di avere infiniti valori possibili, abbiamo un numero finito di livelli (0-4095). Questo introduce il cosiddetto **errore di quantizzazione**.

6. Risoluzione in tensione

Per capire quanto "fine" è la misura possiamo calcolare quanto vale un singolo step dell'ADC:

$$LSB = \frac{V_{Ref}}{(2^N - 1)} = \frac{3.3V}{4095} = 0.806\text{mV}$$

Quindi ogni incremento di 1 nel valore ADC corrisponde a circa 0.806mV. **NB: sulla nostra board il range è 0V – 3.3V.**

7. ADC sulla Nucleo STM32

Nel nostro circuito abbiamo solo un Sample&Hold e un ADC SAR.

Canali ADC

Abbiamo 16 canali esterni + 2 canali interni:

- VREFINT = 1.21V
- Sensore di Temperatura interno (per vedere se il micro è in overheating) oppure Battery Charge Monitor

Far Partire la Conversione

La conversion può partire:

- **Via software**
- **Via Timer (TRGO, Capture and Compare):** il Timer è input dell'ADC, e triggera la conversione.
- **Via Trigger esterno (EXTI):** ad esempio premo un bottone e inizio la conversione.

Modalità di Conversione

- **Polling**: controllando continuamente la EoC flag.
- **Interrupt**: ADC genera una interrupt all'EoC.
- **DMA**: i valori convertiti finiscono direttamente in memoria.

8. Esempio completo

```
uint32_t adc_value;
float voltage;

HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, 100);

adc_value = HAL_ADC_GetValue(&hadc1);
voltage = ((float)adc_value * 3.3f) / 4095.0f;

HAL_ADC_Stop(&hadc1);
```

9. Single Channel/Scanning Mode

1. Single channel

Nel caso più semplice, l'ADC legge **un solo canale**. Se hai un solo sensore, questa è la situazione più semplice.

2. Scanning mode

Supponiamo invece di avere **3 sensori** collegati a 3 ingressi ADC:

- Sensore 1 → Channel 1
- Sensore 2 → Channel 2

- Sensore 3 → Channel 3

Con lo **scanning mode** puoi dire all'ADC:
"Leggi questi canali uno dopo l'altro, secondo un ordine preciso." Ad esempio: [Channel 1 -> Channel 3 -> Channel 2]. Questa è la **sequence**. Quindi non significa semplicemente "scansiona tutti i canali": puoi stabilire **quali canali leggere e in quale ordine**.

3. Regular channels

I **regular channels** sono i canali ADC utilizzati normalmente. Quindi, a seconda dello specifico STM32, puoi avere una sequenza di conversione composta da **fino a 16 canali regular**.
Per esempio:

- Rank 1 → Channel 5
- Rank 2 → Channel 2
- Rank 3 → Channel 8
- Rank 4 → Channel 1

L'ADC eseguirà:
CH5 → CH2 → CH8 → CH1
Il termine **Rank** è importante: indica la **posizione del canale nella sequenza**.

4. Injected channels

Gli **injected channels** sono un secondo gruppo di conversioni che hanno una **priorità maggiore** rispetto alle regular. Quindi possiamo avere, ad esempio:

- REGULAR: CH1 → CH2 → CH3 → CH4
- INJECTED: CH8 → CH9

Se arriva un trigger per gli injected, l'ADC può eseguire la conversione injected con priorità rispetto alla normale sequenza regular.

Perché sono utili?

Immagina di acquisire normalmente dei sensori:
CH1 → CH2 → CH3 → CH4 → CH1 → CH2 → ...
ma improvvisamente arriva un evento importante, per esempio un trigger proveniente da un timer.
Puoi configurare un **injected conversion** per acquisire immediatamente un altro segnale.
In pratica:

- **Regular = acquisizione normale**
- **Injected = acquisizione "speciale", con trigger dedicati e priorità maggiore**

5. Auto-injection

L'idea è che puoi combinare:

- fino a **16 regular channels**
- fino a **4 injected channels**

ottenendo fino a 20 conversioni/canali gestibili nella configurazione complessiva. Con **automatic injection**, le conversioni injected possono essere inserite automaticamente durante la sequenza regular, senza dover necessariamente utilizzare un trigger esterno separato per ogni conversione injected.

6. "Normally all the channels are converted in sequence continuously"

Questa parte è molto importante.
Supponiamo di avere:

- Rank 1 → CH1

- Rank 2 → CH4
- Rank 3 → CH7

La sequenza è CH1 → CH4 → CH7. Se hai configurato la modalità continua, dopo aver terminato la sequenza l'ADC ricomincia. Quindi **non devi avviare manualmente ogni singola conversione**.

7. EoC: End of Conversion

EoC = End of Conversion significa: "Una conversione è terminata". Puoi configurare quando generare questo evento. Ci sono quindi due possibilità concettuali.

EoC per ogni canale

- **CH1 → EoC**
- **CH2 → EoC**
- **CH3 → EoC**

Ogni volta che termina una conversione, hai un EoC.

EoC alla fine della sequenza

CH1 → CH2 → CH3 → EoC

In questo caso aspetti che **tutta la sequenza** sia terminata. Questa distinzione diventa particolarmente importante quando usi **interrupt o DMA**.

8. Discontinuous mode

Questa è probabilmente la parte che crea più confusione.
Normalmente hai una sequenza: CH1 → CH2 → CH3 → CH4 → CH5 → CH6
In modalità normale, quando arriva il trigger, puoi eseguire tutta la sequenza:
Trigger → CH1 → CH2 → CH3 → CH4 → CH5 → CH6
In **discontinuous mode**, invece, puoi dividere la sequenza in **porzioni più piccole**.
Per esempio:
Sequenza completa: CH1 → CH2 → CH3 → CH4 → CH5 → CH6
Potresti configurare 2 conversioni per ogni trigger:

- Trigger 1 → CH1 → CH2
- Trigger 2 → CH3 → CH4
- Trigger 3 → CH5 → CH6

Quindi: **un singolo trigger non fa tutta la sequenza, ma solo una parte della sequenza.**

Concetto	Significato
Single channel	L'ADC converte un solo canale
Scanning mode	L'ADC converte più canali in una sequenza prestabilita
Rank	Posizione di un canale nella sequenza
Regular channel	Canale utilizzato nella normale sequenza ADC
Injected channel	Canale con maggiore priorità e trigger dedicati
Auto-injection	Inserimento automatico delle conversioni injected
Continuous mode	La sequenza viene ripetuta continuamente
EoC	Segnala la fine di una conversione (o della sequenza, a seconda della configurazione)
Discontinuous mode	Un trigger esegue solo una parte della sequenza

10. Conversione singola vs continua

Single conversion

Ogni SoC (Start of Conversion) deve essere dato via software/Timer/External Trigger.
L'ADC fa:

1. Start
2. Conversione
3. Risultato
4. Stop

Poi, se vuoi una nuova misura, devi avviare nuovamente l'ADC (SoC).

Continuous conversion

Lo SoC è generato automaticamente al massimo sampling rate (i.e. l'ADC inizia una nuova conversione appena finisce quella precedente). In questo caso l'ADC continua a campionare il segnale. Questo è molto utile, per esempio, per leggere continuamente un sensore, molto velocemente.

11. Polling, Interrupt e DMA

Questo concetto è molto simile a quello che abbiamo visto per **USART**.
Puoi gestire l'ADC in tre modi principali:

- **Polling:** Il microcontrollore aspetta. È semplice, ma la CPU rimane in attesa.
- **Interrupt:** L'ADC genera un interrupt quando la conversione è terminata. La CPU può continuare a fare altro e viene avvisata quando l'ADC ha terminato.
- **DMA:** Il risultato dell'ADC viene trasferito automaticamente in memoria. Questo è particolarmente utile quando devi acquisire **molti campioni consecutivi**.

12. Right / Left alignment

Il risultato della conversione ADC viene memorizzato in un **registro**, che ha una certa dimensione (tipicamente 16 bit). Se, per esempio, hai un ADC a **12 bit**, il risultato è ADC_Value=0-4095, ma il registro può essere più grande. Puoi decidere come posizionare questi 12 bit nel registro:

Right alignment

I bit vengono messi a destra:



Per esempio: `0000 1010 1100 0011`
È la configurazione più intuitiva perché il valore numerico corrisponde direttamente al numero letto.

Left alignment

I bit vengono spostati verso sinistra:



Per esempio: `1010 1100 0011 0000`
Questo può essere utile in alcune situazioni in cui vuoi utilizzare direttamente i bit più significativi.
In STM32CubeMX questa impostazione appare generalmente come **Data Alignment: Right/Left**.

13. Channel-wise programmable sampling time

Il **sampling time** è il tempo durante il quale l'ADC "osserva" e campiona la tensione in ingresso prima di effettuare la conversione. E soprattutto: **può essere configurato diversamente per ogni canale.**
Ad esempio:

- CH1 → Sampling time = 3 cycles
- CH2 → Sampling time = 15 cycles
- CH3 → Sampling time = 84 cycles

Questo è utile perché sorgenti/sensori diversi possono richiedere tempi di acquisizione diversi.

14. Clock dell'ADC

L'ADC può avere una frequenza di clock massima teorica di 36MHz. Ma poi c'è il problema che 36 MHz non è un divisore intero di 84 MHz, quindi la frequenza di clock massima reale è **21 MHz**.

1. Sampling time + Resolution

La frequenza con cui puoi effettuare le conversioni è legata a:

$$f_{Sample} = \frac{f_{ADC}}{ST + RES}$$

dove:

- f_ADC = frequenza del clock dell'ADC
- ST = **Sampling Time**, espresso in clock cycles
- RES = tempo necessario per la conversione, espresso in clock cycles

Attenzione: qui **RES non significa semplicemente "12 bit"**. Nella formula rappresenta il **numero di cicli di clock necessari alla conversione per quella risoluzione**.

Esempio con ADC a 12 bit

Supponiamo $f_{ADC} = 21\ MHz$, $ST_{min} = 3$ e $RES = 12$.
Quindi:

$$f_{Sample} = \frac{f_{ADC}}{ST + RES} = \frac{21\ MHz}{3 + 12} = 1.4\ MHz$$

Ecco da dove viene: **max sampling frequency = 1.4 MHz**

2. Attenzione: sampling frequency ≠ ADC clock

Questa distinzione è fondamentale. Abbiamo $f_{ADC} = 21\ MHz$, ma **non** significa che otteniamo 21 milioni di campioni al secondo. Per ogni conversione servono diversi cicli di clock. Una conversione richiede $ST + RES$ cicli. Nel nostro esempio:

$$N°\ Campioni\ al\ s\ = \frac{f_{ADC}}{ST + RES} = 1.4 \cdot 10^6\ \frac{campioni}{s}$$

3. Ma perché usare uno ST maggiore?

Se aumentiamo ST diminuisce f_{Sample}, ovvero il numero di campioni al secondo. Perché dovremmo volere aumentarlo quindi? **Perché il condensatore interno di sample-and-hold dell'ADC deve avere abbastanza tempo per caricarsi alla tensione del segnale.**
In particolare, se la sorgente ha un'impedenza elevata, può essere necessario un **sampling time maggiore** per ottenere una misura accurata.

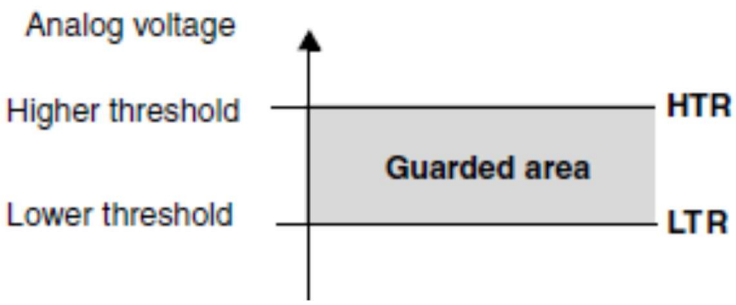
- **Sampling time piccolo → più veloce**
- **Sampling time grande → più tempo per acquisire correttamente il segnale**

15. Analog Watchdog

L'**Analog Watchdog** è una funzionalità molto utile dell'ADC. Permette di controllare automaticamente se il valore misurato è **all'interno di un intervallo stabilito**.

Configuri quindi:

- Lower threshold (LTR)
- Upper threshold = (HTR)



Se il segnale esce dall'intervallo:

- Vin < LTR → WATCHDOG!
- Vin > HTR → WATCHDOG!

L'ADC può quindi generare un **flag/interrupt**. Questo è molto utile per implementare **allarmi, protezioni e controlli di soglia** senza dover continuamente fare il confronto nel codice.

Funzioni

Qui il punto fondamentale è capire che le funzioni si dividono in **3 modalità di acquisizione**:

- **Polling** → Start + PollForConversion + GetValue
- **Interrupt** → Start_IT + callback
- **DMA** → Start_DMA + callback

1. HAL_ADC_Start()

HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef *hadc);

Avvia l'ADC e quindi **inizia la conversione**. hadc è il puntatore alla struttura che identifica l'ADC configurato, per esempio &hadc1. La usi quando vuoi effettuare una conversione ADC in **polling mode**.

```
uint32_t adc_value;

HAL_ADC_Start(&hadc1);

HAL_ADC_PollForConversion(&hadc1, 100);

adc_value = HAL_ADC_GetValue(&hadc1);
```

2. HAL_ADC_Start_IT()

HAL_StatusTypeDef HAL_ADC_Start_IT(ADC_HandleTypeDef *hadc);

Avvia l'ADC utilizzando la modalità **Interrupt**. Quando la conversione termina, invece di dover continuamente controllare se è finita con PollForConversion(), l'ADC genera un **interrupt**. Usala quando vuoi acquisire dati ADC tramite **interrupt**, lasciando alla CPU la possibilità di fare altro mentre l'ADC esegue la conversione.
needed at the beginning of the code also just to setup the peripheral
In alcune configurazioni STM32, HAL_ADC_Start_IT() non serve soltanto per "leggere un valore", ma anche per **abilitare/avviare la gestione dell'ADC tramite interrupt**.

```
HAL_ADC_Start_IT(&hadc1);
```

Quando la conversione sarà terminata, verrà chiamata:

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    uint32_t value;

    value = HAL_ADC_GetValue(hadc);
}
```

3. HAL_ADC_Start_DMA()

HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef *hadc,uint32_t *pData,uint32_t Length);

Avvia l'ADC utilizzando il **DMA**. Il DMA trasferisce automaticamente i risultati dell'ADC dalla periferica alla **RAM**, senza che la CPU debba leggere ogni risultato manualmente.
I parametri sono:

- hadc → ADC da utilizzare
- pData → indirizzo del buffer in RAM dove mettere i risultati
- Length → numero di valori da acquisire

È particolarmente utile quando devi acquisire **molti campioni**.

```
uint32_t adc_buffer[100];

HAL_ADC_Start_DMA(&hadc1, adc_buffer, 100);
```

Significa: Avvia ADC1 e trasferisci automaticamente **100 risultati** nell'array `adc_buffer`. Alla fine avrai:

```
adc_buffer[0]
adc_buffer[1]
adc_buffer[2]
...
adc_buffer[99]
```

4. HAL_ADC_Stop()

HAL_StatusTypeDef HAL_ADC_Stop(ADC_HandleTypeDef *hadc);

Ferma l'ADC e **interrompe la conversione in corso**. Quindi non significa "leggi il risultato e poi fermati": semplicemente **blocca l'acquisizione**. La usi **quando hai terminato l'acquisizione in polling mode e vuoi fermare l'ADC**.

```
HAL_ADC_Start(&hadc1);

HAL_ADC_PollForConversion(&hadc1, 100);

uint32_t value = HAL_ADC_GetValue(&hadc1);

HAL_ADC_Stop(&hadc1);
```

5. HAL_ADC_Stop_IT()

HAL_StatusTypeDef HAL_ADC_Stop_IT(ADC_HandleTypeDef *hadc);

Ferma l'ADC e disabilita la gestione tramite **interrupt**, quando stai utilizzando HAL_ADC_Start_IT() e vuoi terminare l'acquisizione.

6. HAL_ADC_Stop_DMA()

HAL_StatusTypeDef HAL_ADC_Stop_DMA(ADC_HandleTypeDef *hadc);

Ferma:

1. la conversione ADC
2. il trasferimento DMA

Quindi interrompe l'intero processo: ADC → DMA → RAM

Quando hai avviato l'acquisizione con HAL_ADC_Start_DMA() e vuoi interromperla prima che termini normalmente.

7. HAL_ADC_PollForConversion()

HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef *hadc,uint32_t Timeout);

Aspetta che la conversione ADC sia terminata. La parola **polling** significa proprio: "Controllo continuamente se la conversione è terminata." Il parametro `Timeout` indica per quanto tempo massimo aspettare. La usi insieme a HAL_ADC_Start() quando lavori in **polling mode**.

```
HAL_ADC_Start(&hadc1);

if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
{
    uint32_t value = HAL_ADC_GetValue(&hadc1);
}
```

8. HAL_ADC_GetValue()

uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef *hadc);

Restituisce il **risultato digitale della conversione ADC**.

```
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, 100);

adc_value = HAL_ADC_GetValue(&hadc1);
voltage = ((float)adc_value * 3.3f) / 4095.0f;

HAL_ADC_Stop(&hadc1);
```

9. HAL_ADC_ConvCpltCallback()

__weak void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc);

È una **callback** chiamata quando l'acquisizione ADC è stata completata. È particolarmente importante con **Interrupt e DMA**.

```
uint32_t buffer[100];

HAL_ADC_Start_DMA(&hadc1, buffer, 100);

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    // I 100 campioni sono disponibili
    HAL_ADC_Stop_DMA(hadc);
}
```

10. HAL_ADC_ConvHalfCpltCallback()

__weak void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc);

È simile alla precedente, ma viene chiamata quando è stata completata **metà dell'acquisizione DMA**.
Immagina:
`uint32_t buffer[100];`

Quando sono stati acquisiti i primi 50 campioni:
`HAL_ADC_ConvHalfCpltCallback()`

Quando sono stati acquisiti tutti i 100:
`HAL_ADC_ConvCpltCallback()`

È molto utile quando vuoi **elaborare i dati mentre il DMA sta ancora acquisendo**. Questo meccanismo è particolarmente utile per **acquisizioni continue e grandi quantità di dati**.

Progetti

Lab06_1

Lab06_1: acquisisci il voltaggio del potenziometro tramite ADC in polling mode, iniziando la conversione via software e mandando il valore sul seriale del PC ogni secondo.

File .ioc

Pin

- Potentiometer: PA1 = ADC1_IN1
- PA2: USART2_TX
- PA3: USART2_RX

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC: USART2 global interrupt = Enabled

ADC

- Mode: IN1
- Sampling Time: 480 cycles

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Variables

```
uint32_t adc_value=0;
float voltage=0.0;
uint8_t tx_buf[32];
int len;
```

Callbcak Timer (1Hz)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){
		len = sprintf((char*)tx_buf, "Potentiometer: %.2f V\r\n", voltage);
		HAL_UART_Transmit_DMA(&huart2, tx_buf, len);
	}
}
```

Main

```
int main(void)
{

  /…...
  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
	  HAL_ADC_Start(&hadc1);
	  HAL_ADC_PollForConversion(&hadc1, 100);

	  adc_value = HAL_ADC_GetValue(&hadc1);
	  voltage = ((float)adc_value * 3.3f) / 4095.0f;

	  HAL_ADC_Stop(&hadc1);
  }
  /* USER CODE END 3 */
}
```

Lab06_2a

Lab06_2a: acquisisci il voltaggio del potenziometro tramite ADC in **interrupt mode**, mandando il valore sul seriale del PC ogni secondo.

File .ioc

Pin

- Potentiometer: PA1 = ADC1_IN1
- PA2: USART2_TX
- PA3: USART2_RX

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99

- ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled

ADC

- Mode: IN1
- Sampling Time: 480 cycles
- ADC1 global interrupt: Enabled

Codice

Private Include

```
#include "stdio.h"
#include "string.h"
```

Private Variables

```
uint32_t adc_value=0;
float voltage=0.0;
uint8_t tx_buf[32];
int len;
```

Callbacks

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){
		len = sprintf((char*)tx_buf, "Potentiometer: %.2f V\r\n", voltage);
		HAL_UART_Transmit_DMA(&huart2, tx_buf, len);

		// Riarmare l'ADC
		HAL_ADC_Start_IT(&hadc1);
	}
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    // Quando arriva l'EoC aggiorniamo il valore
	  adc_value = HAL_ADC_GetValue(hadc);
	  voltage = ((float)adc_value * 3.3f) / 4095.0f;
}
```

Inizializzazioni (Main, prima del while)

```
HAL_TIM_Base_Start_IT(&htim2);
HAL_ADC_Start_IT(&hadc1);
```

Lab06_2b

Lab06_2b: acquisisci il voltaggio del potenziometro tramite ADC la cui conversione è abilitata da un **trigger a 1 Hz**, mandando il valore sul seriale del PC ogni secondo.

File .ioc

Pin

- Potentiometer: PA1 = ADC1_IN1
- PA2: USART2_TX
- PA3: USART2_RX

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - **Trigger Event Selection (TRGO): Update Event**
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled

ADC

- Mode: IN1
- Sampling Time: 480 cycles
- ADC1 global interrupt: Enabled
- **External Trigger Conversion Source: Timer 2 Trigger Out event**
- **External Trigger Conversion Edge: Trigger detection on the rising edge**

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Variables

```
uint32_t adc_value=0;
float voltage=0.0;
uint8_t tx_buf[32];
int len;
```

Callback ADC

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    // Il Trigger (Timer 2 a 1Hz) avvia la conversione ogni secondo
      // Quando arriva l'EoC aggiorniamo il valore
      adc_value = HAL_ADC_GetValue(hadc);
      voltage = ((float)adc_value * 3.3f) / 4095.0f;

      // E lo inviamo
      len = sprintf((char*)tx_buf, "Potentiometer: %.2f V\r\n", voltage);
      HAL_UART_Transmit_DMA(&huart2, tx_buf, len);

}
```

Inizializzazioni (Main, prima del while)

```
HAL_TIM_Base_Start(&htim2); // Ci serve il Trigger, non una Interrupt
HAL_ADC_Start_IT(&hadc1);
```

Lab06_2c

Lab06_2c: acquisisci il voltaggio del potenziometro tramite ADC la cui conversione è abilitata da un **trigger a 1 Hz**, mandando il valore sullo schermo LCD ogni secondo. La prima riga LCD mostra il valore, la seconda una barra proporzionale al valore.

File .ioc

Pin

Pin	Funzione	Impostazione
PA4	LCD_BL_ON	GPIO_Output
PB1	LCD_E	GPIO_Output
PB2	LCD_RS	GPIO_Output
PB12	LCD_D4	GPIO_Output
PB13	LCD_D5	GPIO_Output
PB14	LCD_D6	GPIO_Output
PB15	LCD_D7	GPIO_Output
PA1	Potentiometer	ADC1_IN

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - **Trigger Event Selection (TRGO): Update Event**
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

ADC

- Mode: IN1
- Sampling Time: 480 cycles
- ADC1 global interrupt: Enabled
- **External Trigger Conversion Source: Timer 2 Trigger Out event**
- **External Trigger Conversion Edge: Trigger detection on the rising edge**

Codice

Private Includes

```
#include "PMDB16_LCD.h"
#include "stdio.h"
#include "string.h"
```

Private Variables

```
uint32_t adc_value=0;
float voltage=0.0;
char string[16];
int bar_value=0;
```

ADC Callback

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    adc_value = HAL_ADC_GetValue(hadc);
```

```
    voltage = ((float)adc_value * 3.3f) / 4095.0f;

    sprintf(string, "Value: %.2f V", voltage);

    bar_value = (int)((voltage / 3.3f) * 80.0f);   // 0-80

    lcd_clear();
    lcd_println(string, 0);   // testo sulla riga superiore
    lcd_drawBar(bar_value);   // barra sulla riga inferiore
}
```

Inizializzazioni (Main, prima del while)

```
  HAL_TIM_Base_Start(&htim2); // Timer 2 (Trigger ADC)
  HAL_ADC_Start_IT(&hadc1);   // ADC (Interrupt Mode)
  lcd_initialize();           // LCD
  lcd_backlight_ON();
```

Lab06_3a

Lab06_3a: l'obiettivo è acquisire 3 tensioni (Potenziometro, Sensore di Temperatura Interno, Vref) ogni secondo e mandarle ad un remote terminal. L'acquisizione viene iniziata via hardware da un timer e i dati vengono salvati nella memoria del microcontrollore tramite **DMA**.

File .ioc

Pin

- Potentiometer: PA1 = ADC1_IN1
- PA2: USART2_TX
- PA3: USART2_RX

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - **Trigger Event Selection (TRGO): Update Event**
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal

- NVIC:
  - DMA stream6 global interrupt = Enabled
  - USART2 global interrupt = Enabled

ADC

- Mode:
  - IN1
  - Temperature Sensor Channel
  - Vrefint Channel
- **DMA2 stream0 global interrupt: Enabled**
- **External Trigger Conversion Source: Timer 2 Trigger Out event**
- **External Trigger Conversion Edge: Trigger detection on the rising edge**
- **Scan Conversion Mode: Enabled**
- **DMA Continuous Request: Enable**
- **N° Conversions: 3**
- **Ranks:**
  - **Rank 1 = Channel 1**
  - **Rank 2 = Temperature Sensor**
  - **Rank 3 = Vref**
  - **ST = 480 cycles**
- **DMA:**
  - **Mode = Circular**
  - **Data Width = Half Word**
  - **Memory Increment**

$$T[°C] = \frac{V_{Sense} - V_{25°C}}{Average\ Slope} + 25°C = \frac{V_{Sense} - 0.76[V]}{2.5\ [mV/°C]} + 25°C$$

Codice

Private Include

```
#include "stdio.h"
#include "string.h"
```

Private Variables

```
uint16_t adc_buf[3];
float v_pot = 0.0f;
float v_temp_sense = 0.0f;
float temperature = 0.0f;
float v_refint = 0.0f;
uint8_t tx_buf[64];
int len;
```

Callback ADC (DMA)

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    // Il Trigger (Timer 2 a 1Hz) avvia la conversione ogni secondo
      // Quando arriva l'EoC aggiorniamo i valori
      v_pot = ((float)adc_buf[0] * 3.3f) / 4095.0f;
      v_temp_sense = ((float)adc_buf[1] * 3.3f) / 4095.0f;
      v_refint = ((float)adc_buf[2] * 3.3f) / 4095.0f;
```

```
	temperature = ((v_temp_sense - 0.76f) / 0.0025f) + 25.0f;

	// E li inviamo
	len = sprintf((char*)tx_buf, "Pot: %.2fV  Temp: %.1fC  Vref: %.2fV\r\n", v_pot,
temperature, v_refint);
	HAL_UART_Transmit_DMA(&huart2, tx_buf, len);
}
```

Inizializzazioni (Main, prima del While)

```
  HAL_TIM_Base_Start(&htim2); // Timer 2 (Trigger ADC)
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buf, 3);   // ADC (DMA Mode), buffer da 3
campioni
```

Lab06_3b

Lab06_3b: mandare ogni secondo il valore della resistenza LDR e della luce in lux ad un remote terminal. Avviare la conversione ADC tramite Timer Trigger e usare l'ADC in modalità DMA.

File .ioc

Pin

- LDR: PA0 = ADC1_IN0
- PA2: USART2_TX
- PA3: USART2_RX

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - **Trigger Event Selection (TRGO): Update Event**
  - PSC=99
  - ARR=839999

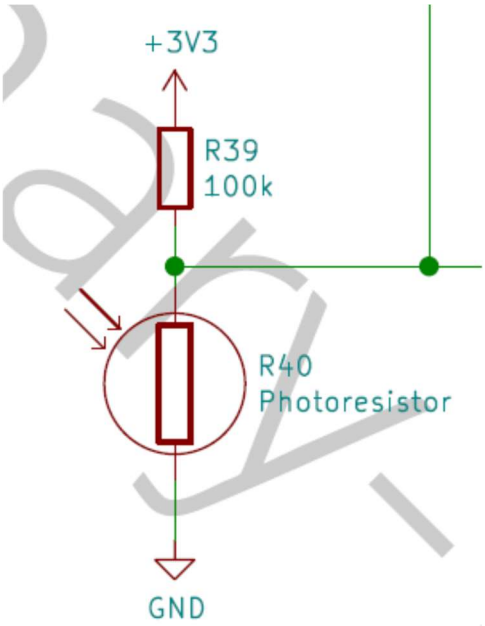
$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled

ADC

- Mode: IN0
- **DMA2 stream0 global interrupt: Enabled**
- **External Trigger Conversion Source: Timer 2 Trigger Out event**
- **External Trigger Conversion Edge: Trigger detection on the rising edge**
- **DMA Continuous Request: Enable**
- **N° Conversions: 1**
- **Ranks:**
  - **Rank 1 = Channel 0**
  - **ST = 480 cycles**
- **DMA:**
  - **Mode = Circular**
  - **Data Width = Half Word**
  - **Memory Increment**



$$V_{ADC} = \frac{LDR}{LDR + 100k} \cdot 3.3V$$

$$LDR = \frac{V_{ADC} \cdot 100k}{3.3 - V_{ADC}}$$

$$Lux = 10 \cdot \left(\frac{100k}{LDR}\right)^{1.25}$$

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
#include <math.h>
```

Private Variables

```
uint16_t adc_buf;
float v_adc = 0.0f;
float ldr = 0.0f;
float lux = 0.0f;
uint8_t tx_buf[64];
int len;
```

ADC Callback DMA

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    // Il Trigger (Timer 2 a 1Hz) avvia la conversione ogni secondo
      // Quando arriva l'EoC aggiorniamo i valori
      v_adc = ((float)adc_buf * 3.3f) / 4095.0f;
      ldr = (v_adc*100)/(3.3-v_adc); //[k ohm]
      lux = 10*pow(100/ldr,1.25); //[lux]



      // E li inviamo
      len = sprintf((char*)tx_buf, "Vadc: %.2fV  LDR: %.1fkOhm  Lux: %.2flux\r\n",
v_adc, ldr, lux);
      HAL_UART_Transmit_DMA(&huart2, tx_buf, len);
}
```

Inizializzazioni (Main, prima del While)

```
  HAL_TIM_Base_Start(&htim2); // Timer 2 (Trigger ADC)
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&adc_buf, 1);  // ADC (DMA Mode), buffer da 1 campione
```

Lab06_3b – versione con Half Complete Callback

Private Includes

```
#include "stdio.h"
#include "string.h"
#include <math.h>
```

Private Variables

```
uint16_t adc_buf[2];   // buffer circolare: [0] pronto a meta' trasferimento, [1] a trasferimento completo
float v_adc = 0.0f;
float ldr = 0.0f;
float lux = 0.0f;
uint8_t tx_buf[64];
int len;
```

Fuori dal Main

```
static void process_and_send(uint16_t raw)
{
    v_adc = ((float)raw * 3.3f) / 4095.0f;
    ldr = (v_adc * 100) / (3.3f - v_adc);       // [kOhm]
    lux = 10 * pow(100 / ldr, 1.25f);            // [lux]

    len = sprintf((char*)tx_buf, "Vadc: %.2fV  LDR: %.1fkOhm  Lux: %.2flux\r\n", v_adc, ldr, lux);
    HAL_UART_Transmit_DMA(&huart2, tx_buf, len);
}

void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
    process_and_send(adc_buf[0]);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    process_and_send(adc_buf[1]);
}
```

Inizializzazioni (Main, prima del While)

```
  HAL_TIM_Base_Start(&htim2); // Timer 2 (Trigger ADC)
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buf, 2);   // buffer da 2 elementi, niente & (l'array decade gia' da solo)
```